SECURITY AUDIT OF

OP_NET Runtime

Public Report

Version: 1.3

March 18, 2026

Verichains Lab

info@verichains.io

https://www.verichains.io

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **March 13, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the OP_NET Runtime smart contract. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit process, the audit team had identified several vulnerable issues in the project.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY . . . 4
1.1. About OP_NET Runtime . . . 4
1.2. Audit Methodology . . . 4
1.3. Disclaimer . . . 4
1.4. Acceptance Minute . . . 5
2. AUDIT OVERVIEW . . . 6
2.1. Audit Scope . . . 6
2.2. Overview . . . 6
3. FINDINGS . . . 9
3.1. `C-01` - `i8` Treated as `u8` Causes Data Corruption `[CRITICAL]` `[FIXED]` . . . 9
3.2. `C-02` - Incorrect Type Size Mapping in `BytesReader` `[CRITICAL]` `[FIXED]` . . . 10
3.3. `C-03` - Underflow in `modInverse()` Function `[CRITICAL]` `[FIXED]` . . . 11
3.4. `C-04` - Incorrect Value Used when Deleting `StorageMap` Item `[CRITICAL]` `[FIXED]` . . . 12
3.5. `C-05` - Incorrect Implementation in P2WPKH Segwit `[CRITICAL]` `[FIXED]` . . . 13
3.6. `C-06` - Critical accuracy loss in preciseLogRatio due to divergent Taylor series `[CRITICAL]` `[FIXED]` . . . 14
3.7. `C-07` - Integer Truncation in Nested `[CRITICAL]` `[FIXED]` . . . 15
3.8. `C-08` - State Corruption in StoredU256 `[CRITICAL]` `[FIXED]` . . . 17
3.9. `H-01` - Index Out Of Bounds in `StorageSet` `[HIGH]` `[FIXED]` . . . 18
3.10. `H-02` - Incorrect Bounds Checking in `StoredPackedArray` `[HIGH]` `[FIXED]` . . . 19
3.11. `M-01` - Missing Offset Bounds Validation in `BytesReader` and `BytesWriter` `[MEDIUM]` `[FIXED]` . 20
3.12. `M-02` - Missing length validation for witnessScript in p2wsh and p2wshOrNull methods `[MEDIUM]` `[FIXED]` . . . 21
3.13. `M-03` - Logarithm Functions Fail to Revert on Zero Input `[MEDIUM]` `[FIXED]` . . . 22
3.14. `M-04` - `defaultValue` of `StoredBoolean` have no effect `[MEDIUM]` `[FIXED]` . . . 23
3.15. `L-01` - Inconsistent Length Check for `subPointer` `[LOW]` `[FIXED]` . . . 24
3.16. `L-02` - Inconsistent Error Handling `[LOW]` `[FIXED]` . . . 25
3.17. `L-03` - Incorrect Key Type in `KeyMerger` `[LOW]` `[FIXED]` . . . 26
3.18. `L-04` - Warning in `isValidWitnessScriptSize` implementation `[LOW]` `[FIXED]` . . . 27
3.19. `L-05` - Missing Cache Optimization in AddressMap and MapUint8Array Deletion `[LOW]` `[FIXED]` . . . 28
3.20. `I-01` - Immutability Protection Bypass in Address Type `[INFO]` `[ACKED]` . . . 29
3.21. `I-02` - Possible Overflow in Runtime Library `[INFO]` `[FIXED]` . . . 30
4. VERSION HISTORY . . . 31

1. MANAGEMENT SUMMARY

1.1. About OP_NET Runtime

The OP_NET Runtime provides the foundational components required for creating smart contracts on Bitcoin Layer 1. Written in AssemblyScript, this runtime allows developers to leverage WebAssembly for efficient contract execution while integrating deeply with Bitcoin's decentralized architecture.

1.2. Audit Methodology

Our security audit process for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using public and our in-house smart contract security analysis tool.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that were considered during the audit of the smart contract:

- Integer Overflow and Underflow
- Timestamp Dependence
- Race Conditions
- Transaction-Ordering Dependence
- DoS with (Unexpected) revert
- DoS with Block Gas Limit
- Gas Usage, Gas Limit and Loops
- Redundant fallback function
- Unsafe type Inference
- Reentrancy
- Explicit visibility of functions state variables (external, internal, private and public)
- Logic Flaws

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and

shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

The audit focus on identifying security flaws in the code of OP_NET Runtime smart contract. It was first conducted on commit `777faca344b8de24b8798e15af5625c39cebbf4d` from git repository `https://github.com/btc-vision/btc-runtime.git`.

This table summarizes the report revision history, mapping each report version to its corresponding Git commit hash.

Report Version	Git Commit
0.1	`777faca344b8de24b8798e15af5625c39cebbf4d`
0.2	`2badc75744885b66c5f2a9bc98649a1db9ea2a79`
0.3	`330beca88368147598ac863982356ff1c3bc99df`
0.4	`7af1b98fc76ad887b7c174e9be3c787dac85c4a3`
0.5	`08fc5f7e9d560146d809fa97a4d36b19aa2cc123`
0.6	`c8583177f1c0a319825e6fba513452057351ba7b`
1.0	`7d848d671430cc0f5d4cc277abd3d9eb52c439d5`
1.1	`49b539a913c9e2db2c348af02a4e75123452c8a0`
1.2	`28eba4259653f712206d5794beae19a0457c2c01`
1.3	`b80fee1ce27b695203baca200b9472bed97d064a`

2.2. Overview

The OP_NET Runtime smart contract provides the foundational components required for creating smart contracts on Bitcoin Layer 1. Written in AssemblyScript, this runtime allows developers to leverage WebAssembly for efficient contract execution while integrating deeply with Bitcoin's decentralized architecture.

The runtime enables direct interaction with Bitcoin L1, allowing developers to create decentralized applications that operate on the Bitcoin network. It utilizes Bitcoin Layer 1 transactions to submit transaction data, ensuring data availability and immutability through Bitcoin's proven infrastructure. The modular architecture provides comprehensive storage management, event handling, and security features that developers can utilize to build everything from simple token contracts to complex applications.

2.2.1. Features

Token Standards

Contracts in OP_NET are AssemblyScript classes that extend base classes. The runtime provides standard token implementations:

- OP20: Fungible token base class similar to ERC-20 with balance tracking, transfers, allowances, mint/burn capabilities, and token metadata
- OP721: Non-fungible token base class with ownership tracking, metadata/URI support, safe transfers, and approval system
- Developers extend these classes to create custom token implementations

Storage System

Storage managed using pointers (`u16`) and sub-pointers (`u256`), encoded and hashed to generate unique storage locations with cryptographic proofs:

- Primitive storage: Integer types (`StoredU8` through `StoredU256`), `StoredBoolean`, `StoredAddress` with lazy loading and change detection
- Arrays: Efficient collections with bit-packing optimizations (256 booleans per slot)
- Strings: Variable-length text with automatic chunking across storage slots
- Maps: Persistent key-value storage (`StoredMapU256`) with automatic state synchronization
- Nested storage: Dynamic allocation with codec-based encoding for complex data structures

Event System

Events allow contracts to emit signals that external observers can listen to for tracking state changes:

- Type-safe definitions through `NetEvent` base class
- ABI-compatible serialization format
- 352-byte maximum payload
- Immutable once emitted
- Used for tracking transfers, state changes, and contract interactions

Bitcoin Integration

Deep integration with Bitcoin's native functionality:

- Address support: All Bitcoin formats (P2PKH, P2SH, Bech32) with network-specific validation
- Script operations: Parse bytecode, handle opcodes, support SegWit transactions
- Blockchain context: Access block data, transaction details, and UTXO information

Security Features

Security features built into the runtime:

- SafeMath: Automatic overflow/underflow protection for all arithmetic operations (addition, subtraction, multiplication, division, modulo, power, square root) supporting U256, U128, and U64 types
- Reentrancy protection: `ReentrancyGuard` with STANDARD mode (prevents all reentrant calls) and CALLBACK mode (allows controlled reentrancy)
- Type safety: AssemblyScript compile-time type checking catches errors before deployment

Blockchain Environment

The `Blockchain` object is the backbone of the OP_NET runtime, providing essential functionality:

- Contract state management and transaction handling
- Block and transaction data access via `Potential<T>` lazy initialization
- Storage operations (persistent and transient) using `setStorageAt` and `getStorageAt` methods
- Cross-contract calls through `callContract()` function
- Event emission system
- Cryptographic operations (SHA-256 hashing, Schnorr signature verification)
- Network detection and plugin system

Serialization and Buffer Management

Efficient data handling for contract operations:

- BytesReader: Parse calldata and decode typed data with automatic offset tracking
- BytesWriter: Encode return values and serialize for storage with dynamic buffer growth
- Function selector encoding via SHA-256 hashing
- Support for all standard types plus arrays, nested structures, and length-prefixed strings

2.2.2. Architecture

Pointer-Based Storage

Storage pointers and sub-pointers are encoded and hashed to create unique and secure storage locations:

- Primary pointers (`u16`) for namespace allocation
- Sub-pointers (`u256`) for specific data within a namespace
- Encoded and hashed to generate unique storage locations
- Prevents storage collisions between contracts
- Ensures data integrity and prevents tampering through cryptographic proofs
- Managed using `Blockchain.setStorageAt` and `Blockchain.getStorageAt` methods

Singleton Pattern

The `Blockchain` object uses a singleton pattern ensuring consistent state management and providing centralized access to blockchain functionality throughout contract execution.

Lazy Loading

Storage values are only loaded from blockchain when accessed, minimizing gas costs for operations that don't need all data.

Modular Design

The architecture is highly modular:

- Core modules: environment, contracts, storage, events, types
- Supporting modules: buffer, math, memory, script, cryptography
- Plugin system for extensibility
- Shared libraries for common patterns

3. FINDINGS

3.1. `C-01` - `i8` Treated as `u8` Causes Data Corruption `[CRITICAL]` `FIXED`

SCOPE

- Commit: `2badc75744885b66c5f2a9bc98649a1db9ea2a79`
- Files: `runtime/buffer/BytesWriter.ts#write<T>()`

DESCRIPTION

The generic `write<T>()` methods incorrectly handle signed 8-bit integers (`i8`) by treating them as unsigned 8-bit integers (`u8`), causing sign extension vulnerabilities and silent data corruption during serialization.

```
public write<T>(value: T): void {
    if (isInteger<T>()) {
        const size = sizeof<T>();
        if (size === 1) {
            this.writeU8(<u8>value); // BUG: Casts i8 to u8, losing sign info
            return;
        }
        // ...
    }
}
```

RECOMMENDATION

Add proper `i8` handling with dedicated `writeI8()` method.

3.2. `C-02` - Incorrect Type Size Mapping in `BytesReader` `[CRITICAL]` `FIXED`

SCOPE

- Commit: 2badc75744885b66c5f2a9bc98649a1db9ea2a79
- Files: `runtime/buffer/BytesReader.ts`

DESCRIPTION

The generic `read<T>()` method in `BytesReader.ts` contains a critical bug in its integer deserialization logic. The method incorrectly maps byte sizes to read operations, causing data corruption when processing multi-byte integers.

```
 if (isSigned<T>()) {
    const size = sizeof<T>(); // @auditor: return size of T in bytes
    switch (size) {
        case 8: // @auditor: 8 bytes, but read as 1 byte (8 bits)
            return this.readU8() as T;
        case 16: // @auditor: 16 bytes, but read as 2 bytes (16 bits)
            return this.readI16() as T;
        //... and so on...
    }
} else {
    const size = sizeof<T>();
    switch (size) {
        case 8: // @auditor: 8 bytes, but read as 1 byte (8 bits)
            return this.readU8() as T;
        case 16: // @auditor: 16 bytes, but read as 2 bytes (16 bits)
            return this.readU16() as T;
        //... and so on...
    }
}
```

RECOMMENDATION

Use the correct byte size mapping.

UPDATES

This issue was fixed in commit `330beca88368147598ac863982356ff1c3bc99df` .

3.3. C-03 - Underflow in modInverse() Function [CRITICAL] FIXED

SCOPE

- Commit: 330beca88368147598ac863982356ff1c3bc99df
- Files: runtime/types/SafeMath.ts#modInverse()

DESCRIPTION

The modInverse function incorrectly calculates the modular multiplicative inverse due to an integer underflow vulnerability.

The function implements the Extended Euclidean Algorithm to find the modular inverse. In this algorithm, the coefficient s and r (as well as old_s and old_r) are updated in each step of the loop. The value of this coefficient can become negative during the calculation.

However, the variables are declared as u256 , an **unsigned integer** type, which cannot represent negative values. When the subtraction results in a negative number, an integer underflow occurs. This causes variable to wrap around to a very large positive value, corrupting the calculation. Consequently, the function returns a mathematically incorrect result instead of the true modular inverse.

```
public static modInverse(k: u256, p: u256): u256 {
  //...
  r = u256.sub(old_r, u256.mul(quotient, r)); // unchecked subtract
  // ...
  s = u256.sub(old_s, u256.mul(quotient, s)); // unchecked subtract
  // ...
}
```

This bug will cause any cryptographic or mathematical operations that rely on this function to fail, potentially leading to transaction reverts, incorrect signature verifications, or broken mathematical protocols.

RECOMMENDATION

Use the correct data type in calculations.

UPDATES

The OP_NET team has acknowledged this issue and addressed it by implementing a new approach for this SafeMath operation.

3.4. C-04 - Incorrect Value Used when Deleting StorageMap Item [CRITICAL] FIXED

SCOPE

- Commit: 330beca88368147598ac863982356ff1c3bc99df
- Files: runtime/nested/storage/StorageMap.ts

DESCRIPTION

The code expects 32-byte aligned storage slots (new Uint8Array(32) , or EMPTY_BUFFER). However, The delete() method stores new Uint8Array(0) instead of the required 32-byte buffer, breaking future has() checks. The has() calls may return incorrect results due to storage format inconsistency, breaking map semantics and causing data integrity issues.

```
public delete(key: K): bool {
    const storageKey = this.getStorageKey(key);
    if (!Blockchain.hasStorageAt(storageKey)) {
        return false;
    }
    Blockchain.setStorageAt(storageKey, new Uint8Array(0)); // @auditor: wrong length of Uint8Array
    return true;
}
public hasStorageAt(pointerHash: Uint8Array): bool {
    // We mark zero as the default value for the storage, if something is 0, the storage slot get
deleted or is non-existent
    const val: Uint8Array = this.getStorageAt(pointerHash);
    return !eqUint(val, EMPTY_BUFFER);
}
// runtime/math/bytes.ts
export const EMPTY_BUFFER: Uint8Array = new Uint8Array(32);
```

RECOMMENDATION

Use the defined EMPTY_BUFFER for consistency.

```
Blockchain.setStorageAt(storageKey, EMPTY_BUFFER);
```

UPDATES

This issue has been fixed in commit b817c28fc7eb2fdf1be1efa17531221f34417cc1 .

3.5. C-05 - Incorrect Implementation in P2WPKH Segwit [CRITICAL] FIXED

SCOPE:

- Commit: acd8df6bbc072e59ac23678462c5720e7aa9092c
- Files: runtime/script/Segwit.ts

DESCRIPTION

The current implementation of p2wpkh (Pay-to-Witness-Public-Key-Hash) method in Segwit.ts supports uncompressed public keys (pubkeys with length of 65 bytes). However, this is an unsupported implementation that has been flagged as problematic in several documents. For reference, the team can review the warning in the witness field section of the document linked below: https://learnmeabitcoin.com/technical/script/p2wpkh

> Only compressed public keys are accepted in P2WPKH. So when you create the public key hash for the ScriptPubKey, make sure it's the hash of a compressed public key and not an uncompressed public key, otherwise it will be considered non-standard and will not be relayed by nodes (although it's still technically valid and could get mined into the blockchain if you can send it directly to a miner). This is different to P2PKH, where uncompressed public keys are allowed.

Due to this implementation error, the UTXO remains locked with the user's scriptPubkey but cannot be unlocked unless the nodes update their mechanism.

```
// runtime/script/Segwit.ts
public static p2wpkh(hrp: string, pubkey: Uint8Array): string {
    if (pubkey.length !== 33 && pubkey.length !== 65) {
        throw new Revert('Public key must be 33 bytes (compressed) or 65 bytes (uncompressed)');
    }
    // P2WPKH uses HASH160 of the public key as the witness program
    const program = hash160(pubkey);
    return Bech32.encode(hrp, 0, program);
}
```

RECOMMEDATION

Restrict the pubkey to only accept compressed public keys with a length of 33 bytes

UPDATE

This issue was fixed in commit 08fc5f7e9d560146d809fa97a4d36b19aa2cc123 . The p2wpkh function was updated to restrict the size of pubkey.

3.6. C-06 - Critical accuracy loss in preciseLogRatio due to divergent Taylor series [CRITICAL] FIXED

SCOPE

- Commit: a04544324b3149b19368eb0c3dbc4c68f7a76f9f
- Files: runtime/types/SafeMath.ts .

DESCRIPTION

The preciseLogRatio(a, b) function calculates the natural logarithm of a ratio using a helper function, calculateLnOnePlusFraction. This helper implements a 5-term Taylor series approximation ($\ln(1+x) \approx x - x^2/2 + ...$) centered at $x = 0$.

While this approximation is accurate for ratios close to 1 (where $x \approx 0$), it diverges significantly as the normalized ratio approaches 2 (where $x \to 1$).

For example, given a ratio of 1.99 (a=199 , b=100):

- True value: ~688,134 (scaled by 10^6)
- Returned value: 773,432
- Error: ~85,298 units (**~12.4%**)

This error margin makes the function unsuitable for financial mathematics or bonding curves, contradicting the library's design goal of high precision. Additionally, this creates inconsistent results when comparing preciseLogRatio(a, b) against preciseLog(a) - preciseLog(b).

RECOMMENDATION

Replace the usage of calculateLnOnePlusFraction with the existing SafeMath.polyLn1p3 function.

polyLn1p3 utilizes a hyperbolic arctangent transformation ($w = \frac{z}{2+z}$), which guarantees rapid convergence and high precision (± 6 units) across the entire input domain $[0, 1)$. The redundant calculateLnOnePlusFraction helper should subsequently be removed.

UPDATE

This issue was fixed in commit 2b81343470c90b5b5b0e9fc6aa226d67f9dd4b0a .

3.7. C-07 - Integer Truncation in Nested [CRITICAL] FIXED

SCOPE

- Commit: f712cb75e5be4071a3eeee378829317dfc5f87bf
- Files: runtime/memory/Nested.ts:L64

DESCRIPTION

The toValue method, used to decode data from storage back into the generic type T , contains an implementation error for integer types. It only retrieves and returns the first byte (value[0]) of the stored data.

```
private toValue(value: Uint8Array): T {
    // Check T's compile-time type ID
    if (idof<T>() == idof<u256>()) {
        // We know T is u256
        return changetype<T>(u256.fromUint8ArrayBE(value));
    } else if (idof<T>() == idof<Uint8Array>()) {
        // We know T is Uint8Array
        return changetype<T>(value);
    } else if (idof<T>() == idof<Address>()) {
        // We know T is Address
        return changetype<T>(value);
    } else if (isInteger<T>()) {
        // For a simple integer, just pull out the first byte
        return value[0] as T;  //@Auditor: <-- Mistake when T equals u32 or u64
    } else if (isString<T>()) {
        // T is a string
        return changetype<T>(String.UTF8.decode(value.buffer));
    }
    throw new Revert('Unsupported type');
}
```

This decoding logic is inconsistent with the from method below, which correctly handles multi-byte integers. By only returning value[0] , toValue fails to reconstruct the full values that from has encoded.

```
private from(value: T): Uint8Array {
    if (isReference<T>()) {
        if (idof<T>() == idof<u256>()) {
            // Cast T to u256, then convert to bytes
            return changetype<u256>(value).toUint8Array(true);
        } else if (idof<T>() == idof<Uint8Array>()) {
            // Just return it
            return changetype<Uint8Array>(value);
        } else if (idof<T>() == idof<Address>()) {
            // Address is already bytes
            return changetype<Uint8Array>(value);
        }
    }
    else if (isInteger<T>()) {
        const writer = new BytesWriter(sizeof<T>());
        writer.write<T>(value);
        return writer.getBuffer();
    } else if (isString<T>()) {
        const str = changetype<string>(value);
        return Uint8Array.wrap(String.UTF8.encode(str));
    }
    throw new Revert('Unsupported type');
}
```

RECOMMENDATION:

Refactor toValue to use BytesReader and sizeof<T>() to decode the full integer length based on the actual type T .

UPDATE

This issue was fixed in commit `58ab5805f4f4612c68105c513ec1f5a65b223ff5`

3.8. C-08 - State Corruption in StoredU256 [CRITICAL] FIXED

SCOPE

- Commit: `f712cb75e5be4071a3eeee378829317dfc5f87bf`
- Files: `runtime/storage/StoredU256.ts`
- Affected Functions: `set value`

DESCRIPTION

The `value` setter in `StoredU256` compares the new value against the uninitialized cache (`this._value`) instead of the actual storage value. When the cached value is `0` (default) and the caller attempts to set the storage to `0` , the function returns early without writing to storage, even if the actual storage contains a non-zero value.

This creates a state corruption scenario where storage updates are silently ignored.

```
public set value(value: u256) {
    if (u256.eq(value, this._value)) {  // @Auditor: Compares with unloaded cache
        return;
    }
    this._value = value;
    Blockchain.setStorageAt(this.pointerBuffer, this.__value);
}
```

RECOMMENDATION

Call `ensureValue()` before comparison, or use the getter `this.value` which internally loads from storage:

```
public set value(value: u256) {
    if (u256.eq(value, this.value)) {  //@Auditor: Calls getter -> ensureValue() -> loads storage
        return;
    }
    this._value = value;
    Blockchain.setStorageAt(this.pointerBuffer, this.__value);
}
```

UPDATE

This issue was fixed in commit `b63002fa96d05477ec8388b1f227a0b8b7e81828`

3.9. `H-01` - Index Out Of Bounds in `StorageSet` `[HIGH]` `FIXED`

SCOPE

- Commit: `330beca88368147598ac863982356ff1c3bc99df`
- Files: `runtime/nested/storage/StorageSet.ts`

DESCRIPTION

```
public add(value: T): void {
    const storageKey = this.getStorageKey(value);
    // A 1-byte array with a single 0x01 is enough to mark presence
    const flag = new Uint8Array(1);
    flag[31] = 1; // @auditor: invalid index accessing
    Blockchain.setStorageAt(storageKey, flag);
}
```

The `flag` array was allocated with a size of 1 but accessed out of bounds at index 31.

RECOMMENDATION

Use the correct array initializing.

UPDATES

This issue has been fixed in commit `b817c28fc7eb2fdf1be1efa17531221f34417cc1` .

3.10. `H-02` - Incorrect Bounds Checking in `StoredPackedArray` `[HIGH]` `FIXED`

SCOPE

- Commit: 330beca88368147598ac863982356ff1c3bc99df
- Files:
  - `runtime/storage/arrays/StoredPackedArray.ts`

DESCRIPTION

Incorrect bounds checking uses `>` instead of `⩾` allowing access to boundary index (MAX_LENGTH) which should be out of bounds, potentially causing buffer overflow and storage corruption.

```
if (index > this.MAX_LENGTH) {  //  @auditor: Allows index = MAX_LENGTH
    throw new Revert('get: out of range');
}
```

RECOMMENDATION

Change all bounds checks from `index > MAX_LENGTH` to `index ⩾ MAX_LENGTH` to prevent out-of-bounds access.

UPDATES

This issue has been fixed in commit `b817c28fc7eb2fdf1be1efa17531221f34417cc1` .

3.11. `M-01` - Missing Offset Bounds Validation in `BytesReader` and `BytesWriter` `[MEDIUM]` `FIXED`

SCOPE

- Commit: `777faca344b8de24b8798e15af5625c39cebbf4d`
- Files:
  - `runtime/buffer/BytesWriter.ts#setOffset()`
  - `runtime/buffer/BytesReader.ts#setOffset()`

DESCRIPTION

Both `BytesReader` and `BytesWriter` classes allow setting arbitrary offsets without validating that the new offset is within buffer bounds. This creates different security risks for each class:

- `BytesReader` : Setting negative offsets can cause buffer underflow when subsequent read operations access memory outside the allocated buffer, potentially leading to data corruption, transaction reverts, or unintended behavior.
- `BytesWriter` : The class intentionally prohibits buffer resizing (the `resize()` method always throws a Revert), but the `setOffset()` method allows setting offsets beyond the current buffer length without validation. This creates a delayed failure scenario where:
  - Users can successfully call `setOffset(largeValue)` without immediate error
  - The invalid state persists until the next write operation
  - Any subsequent `writeU8()` , `writeU32()` , etc. will trigger `allocSafe()`
  - `allocSafe()` detects the buffer overflow and calls `resize()`
  - The `resize()` method immediately reverts the entire transaction

```
// BytesReader.ts - No validation
public setOffset(offset: i32): void {
    this.currentOffset = offset; // @auditor: Can set negative or out-of-bounds offset
}
// BytesWriter.ts - No validation
public setOffset(offset: u32): void {
    this.currentOffset = offset; // @auditor: Can set offset beyond buffer length
}
```

RECOMMENDATION

Add bounds validation to ensure offset is within valid range.

UPDATES

This issue was fixed in commit `330beca88368147598ac863982356ff1c3bc99df` by:

- adding a check in BytesReader
- removing the function in BytesWriter

3.12. M-02 - Missing length validation for witnessScript in p2wsh and p2wshOrNull methods [MEDIUM] FIXED

SCOPE

- Commit: `acd8df6bbc072e59ac23678462c5720e7aa9092c`
- Files: `runtime/script/Segwit.ts`

DESCRIPTION

In both p2wsh and p2wshOrNull methods, the code does not verify the length of witnessScript to ensure it complies with the witness script limit. If a user locks a UTXO with a witness script exceeding the standard limit, the UTXO can be locked successfully. However, this oversized witness script cannot be used to unlock that UTXO.

```
// runtime/script/Segwit.ts
public static p2wsh(hrp: string, witnessScript: Uint8Array): string {
    // P2WSH uses SHA256 of the script as the witness program
    const program = sha256(witnessScript);
    return Bech32.encode(hrp, 0, program);
}
/**
 * Safe version of p2wsh that returns null on error
 * This is useful when you need to handle encoding failures gracefully
 */
public static p2wshOrNull(hrp: string, witnessScript: Uint8Array): string | null {
    const program = sha256(witnessScript);
    return Bech32.encodeOrNull(hrp, 0, program);
}
```

RECOMMEDATION

Implement length validation to ensure the witnessScript remains below the standard limit.

UPDATES

This issue was fixed in commit `08fc5f7e9d560146d809fa97a4d36b19aa2cc123` . Both the functions was updated to check size of witNessScript before encoding.

3.13. M-03 - Logarithm Functions Fail to Revert on Zero Input [MEDIUM] FIXED

SCOPE

- Commit: `c8583177f1c0a319825e6fba513452057351ba7b`
- Files: `runtime/types/SafeMath.ts`
- Affected Functions: `approximateLog2` , `preciseLog` , `approxLog`

DESCRIPTION

The `approximateLog2` , `preciseLog` , and `approxLog` functions incorrectly return `0` when the input is zero. Mathematically, the logarithm of zero is `undefined` . This behavior creates a collision where input `0` and input `1` both produce the result `0` , allowing empty values to be treated as unit values in calculations.

```
public static preciseLog(x: u256): u256 {
    if (x.isZero() || u256.eq(x, SafeMath.ONE)) {
        return u256.Zero;
    }
...
```

RECOMMENDATION

Update all three functions to revert (throw an error) when the input is zero, correctly enforcing the mathematical undefined state for $\log(0)$.

UPDATE

This issues was fixed in commit `be867983166f4cf47e042072ec8bb3c1dec791b7` . These functions check and revert when input equals zero

3.14. M-04 - defaultValue of StoredBoolean have no effect [MEDIUM] FIXED

SCOPE

- Commit: 59214ce5a7022aa293db5d552bb396b800ccee5c
- Files: runtime/storage/StoredBoolean.ts

DESCRIPTION

```
export class StoredBoolean {
    constructor(
        public pointer: u16,
        defaultValue: bool,
    ) {
        const value = GET_EMPTY_BUFFER();
        if (defaultValue) {
            value[0] = 1;
        }
        this._value = value;
    }
    public get value(): bool {
        this.ensureValue();
        return this._value[0] === 1;
    }
    public set value(value: bool) {
        this._value[0] = value ? 1 : 0;
        Blockchain.setStorageAt(this.pointerBuffer, this._value);
    }
    public toUint8Array(): Uint8Array {
        if (!this._value) {
            throw new Revert(`Not defined.`);
        }
        return this._value;
    }
    private ensureValue(): void {
        this._value = Blockchain.getStorageAt(this.pointerBuffer);
    }
```

In the constructor, StoredBoolean initializes this._value with the default value but does not persist it to storage (unlike the value() setter). Meanwhile, the value() getter calls ensureValue() to load data from storage, which overwrites the in-memory default. As a result, the default value is effectively ignored.

This can also lead to data inconsistency, since toUint8Array() reads directly from this._value without referencing the value stored on-chain.

RECOMMENDATION

Update the constructor:

```
constructor(
    public pointer: u16,
    defaultValue: bool,
) {
    this.pointerBuffer = encodePointer(pointer, EMPTY_POINTER, true, 'StoredBoolean');
    this._value = GET_EMPTY_BUFFER();
    this.value = defaultValue;
}
```

UPDATES

Fixed in commit b80fee1ce27b695203baca200b9472bed97d064a .

3.15. L-01 - Inconsistent Length Check for subPointer [LOW] FIXED

SCOPE

- **Commit**: 777faca344b8de24b8798e15af5625c39cebbf4d
- **Files**:
  - runtime/storage/StoredAddress.ts#L15
  - runtime/storage/StoredU256.ts#L16
  - runtime/storage/StoredU32.ts#L38
  - runtime/storage/StoredU64.ts#L38

DESCRIPTION

The following code is the implementation of encodePointer function in runtime/math/abi.ts :

```
export function encodePointer(uniqueIdentifier: u16, typed: Uint8Array, safe: boolean = true):
Uint8Array {
    const array = ensureAtLeast30Bytes(typed);
    if (safe) assert(array.length === 30, `Pointers must be 30 bytes. Got ${array.length}.`);
    const finalPointer = new Uint8Array(32);
    finalPointer[0] = uniqueIdentifier & 0xff;
    finalPointer[1] = (uniqueIdentifier >> 8) & 0xff;
    for (let i = 0; i < 30; i++) {
        finalPointer[i + 2] = array[i];
    }
    return finalPointer;
}
```

Several classes uses subPointer.length ⩽ 30 to validate subPointer length, which allows arrays shorter than 30 bytes to pass. However, encodePointer expects exactly 30 bytes in safe mode (safe ⩵ true), so this inconsistency could lead to unexpected behavior or errors if shorter arrays are provided.

For example, in StoredU256.ts :

```
/**
 * @constructor
 * @param {u16} pointer - The primary pointer identifier.
 * @param {Uint8Array} subPointer - The sub-pointer for memory slot addressing.
 */
constructor(
    public pointer: u16,
    public subPointer: Uint8Array,
) {
    assert(subPointer.length <= 30, `You must pass a 30 bytes sub-pointer. (StoredU64, got
${subPointer.length})`);
    this.bufferPointer = encodePointer(pointer, subPointer);
}
```

RECOMMENDATION

Update length checks to consistently enforce exactly 30 bytes for subPointer , matching the expectations of encodePointer . Alternatively, consider removing these checks and relying on the checks implemented in abi.ts for encodePointer .

UPDATES

This issue was fixed in commit da8e4b55d2b77cb70c22f307823578e6557c5ef3 by updating the checks to assert(subPointer.length ⩵ 30, ...)

3.16. `L-02` - Inconsistent Error Handling `[LOW]` `FIXED`

SCOPE

- Commit: `777faca344b8de24b8798e15af5625c39cebbf4d`
- Files: `runtime/`

DESCRIPTION

The codebase exhibits inconsistent error handling patterns with mixed usage of `Revert` , and `Error` types across similar operations and contexts. This inconsistency creates unclear error semantics and makes it difficult to distinguish between recoverable business logic failures and unrecoverable system errors.

```
// BytesWriter.ts - Uses Revert for type validation
throw new Revert(`Unsupported integer size: ${size}`);
throw new Revert('Array size is too large');

// BytesReader.ts - Uses Error for similar type validation
throw new Error(`Invalid size ${size}`);
throw new Error(`Unsupported type ${id}`);

// Address.ts - Mixed usage in same class
throw new Error('Invalid public key length');           // Error
throw new Revert('Cannot modify address data.');        // Revert

// bytes.ts - Mixed usage for input validation
throw new Error('bytesToU32: input must be at least 4 bytes');  // Error
throw new Revert('Bit index out of range');                     // Revert
```

RECOMMENDATION

Conduct systematic review and standardization of error types across all runtime files to ensure semantic consistency.

UPDATES

The issue was fixed on commit `63b639ee3c3c8b43986c86bfbcf7ed9f28433bc5` by using `Revert` instead of `Error` for throwing.

3.17. `L-03` - Incorrect Key Type in `KeyMerger` `[LOW]` `FIXED`

SCOPE

- Commit: `2badc75744885b66c5f2a9bc98649a1db9ea2a79`
- Files: `runtime/memory/KeyMerger.ts`

DESCRIPTION

The type of children key is K2, but some functions use `K` or `string` .

```
export class KeyMerger<K extends string, K2 extends string, V extends Uint8Array> {
    public parentKey: K;
    // ...
    public set(key2: K2, value: V): this {
        // ...
    }
    public get(key: K): Uint8Array { // @auditor: key should be K2
        // ...
    }
    public has(key: K): bool { // @auditor: key should be K2
        // ...
    }
    public delete(_key: K): bool { // @auditor: _key should be K2
        // ...
    }
    private mergeKey(key: string): string { // @auditor: key should be K2
        // ...
    }
    private encodePointer(key: string): Uint8Array { // @auditor: key should be K2
        // ...
    }
}
```

RECOMMENDATION

Use the correct type for key.

UPDATES

This issue was fixed in commit `330beca88368147598ac863982356ff1c3bc99df` .

3.18. `L-04` - Warning in `isValidWitnessScriptSize` implementation `[LOW]` `FIXED`

SCOPE

- Commit: `acd8df6bbc072e59ac23678462c5720e7aa9092c`
- Files: `runtime/script/BitcoinCodec.ts`

DESCRIPTION

The witnessScript size verification currently follows the BIP0141 limit of 10,000 bytes. However, when the size exceeds 3,600 bytes, it's marked as non-standard, and the transaction will not be relayed by nodes.

```
public static isValidWitnessScriptSize(witnessScript: Uint8Array): bool {
    return witnessScript.length <= 10000;
}
```

This issue has been documented in some resources. For reference, the team can review:

- https://learnmeabitcoin.com/technical/script/p2wsh/ in the limit section
- https://github.com/bitcoin/bitcoin/blob/master/src/policy/policy.h#L56-L57

RECOMMEDATION

Issue a warning when the witnessScript exceeds 3600 bytes.

UPDATE

This issue was fixed in commit `08fc5f7e9d560146d809fa97a4d36b19aa2cc123` . The isValidWitnessScript-Size function was added a description to warning the witnessScript size.

3.19. L-05 - Missing Cache Optimization in AddressMap and MapUint8Array Deletion [LOW] FIXED

SCOPE

- Commit: c8583177f1c0a319825e6fba513452057351ba7b
- Files: runtime/generic/AddressMap.ts , runtime/generic/MapUint8Array.ts

DESCRIPTION

The delete function in AddressMap fails to utilize the internal _lastIndex cache optimization, forcing an expensive O(N) linear scan for every deletion operation. This inconsistency with the set method (which correctly checks the cache) leads to unnecessary gas consumption and CPU cycle waste, especially when deleting recently accessed or inserted keys.

Unlike set , which performs a fast-path check against _lastIndex , delete immediately delegates to indexOf , which iterates through the entire key array.

```
public delete(key: Address): bool {
    // ISSUE: Missing cache check loop found in set()
    const index = this.indexOf(key); // Forces O(N) scan
    if (index === -1) return false;

    ...
    return true;
}
```

RECOMMENDATION

Implement the same cache verification logic used in set at the beginning of the delete function to enable O(1) deletion for cached keys.

UPDATE

This issue was resolved in commit 8d6d87fbcf845b407cb93621152c79ba216f9428

3.20. `I-01` - Immutability Protection Bypass in Address Type `[INFO]` `ACKED`

SCOPE

- Commit: 330beca88368147598ac863982356ff1c3bc99df
- Files: `runtime/types/Address.ts`

DESCRIPTION

The `Address` class implements partial immutability protection through the `isDefined` flag in its `___set()` operator and `newSet()` method. However, this protection can be completely bypassed using inherited `Uint8Array` methods like `set()` and `fill()` , allowing developers to modify supposedly immutable address data.

```
export class Address extends Uint8Array {
    //...
}
declare class Uint8Array extends TypedArray<u8> {
   //...
}
declare abstract class TypedArray<T> implements ArrayBufferView {
  fill(value: T, start?: i32, end?: i32): this; // @auditor: Fills the array with a specific value
  set<U extends ArrayLike<number>>(source: U, offset?: i32): void // @auditor: Copies values from an
array-like source
}
```

RECOMMENDATION

Implement comprehensive protection by overriding all inherited mutating methods.

3.21. I-02 - Possible Overflow in Runtime Library [INFO] FIXED

SCOPE

- Commit: c8583177f1c0a319825e6fba513452057351ba7b
- Files:
  - runtime/storage/BasedStoredString.ts
  - runtime/buffer/BytesWriter.ts
  - runtime/storage/arrays/StoredPackedArray.ts

DESCRIPTION

Several potential integer overflow points were identified in the codebase. While these issues are currently unlikely to manifest due to constrained data sizes, they require careful consideration during system integration and must be proactively addressed in future development to prevent latent risks.

```
// @auditor:  Same pattern in BasedStoredString
// File: runtime/storage/BasedStoredString.ts
chunkCount += (remaining + 32 - 1) / 32;
// @auditor: Identical overflow vulnerability in string chunk calculation

// @auditor: Buffer offset overflow in writeRaw()
// File: runtime/buffer/BytesWriter.ts
this.currentOffset = off + n;
// @auditor: No overflow check: if `off + n > u32.MAX_VALUE`, wraps to small value

// @auditor: Range operation index overflow
// File: runtime/storage/arrays/StoredPackedArray.ts
const end = startIndex + <u32>values.length;
this.set(startIndex + <u32>i, values[i]);
// @auditor: startIndex + length can overflow, corrupting adjacent data
```

RECOMMENDATION

Ensure that variable lengths and maximum values are checked before performing any arithmetic increases.

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
0.1	*Aug 27, 2025*	Draft Report	Verichains Lab
0.2	*Oct 14, 2025*	Draft Report	Verichains Lab
0.3	*Nov 21, 2025*	Draft Report	Verichains Lab
0.4	*Nov 25, 2025*	Draft Report	Verichains Lab
0.5	*Dec 29, 2025*	Draft Report	Verichains Lab
0.6	*Jan 16, 2026*	Draft Report	Verichains Lab
1.0	*Jan 19, 2026*	Private Report	Verichains Lab
1.1	*Feb 02, 2026*	Private Report	Verichains Lab
1.2	*Mar 13, 2026*	Public Report	Verichains Lab
1.3	*Mar 18, 2026*	Public Report	Verichains Lab